Detecting Machine-Generated Text --- Project Code

Course Natural Language Processing, Final-Term Project, Summer 2025-2026 **Group** 02, Section B **Task** binary classification, human-written against machine-generated text **Corpora** DAIGT V2 (D1) and HC3 (D2)

This notebook holds the four pieces of code the report appendix asks for, in order.

1. Data preprocessing
2. BERT at its best configuration
3. DeBERTa at its best configuration, the BERT variant
4. The soft-vote ensemble of the two

Everything runs at **full corpus scale**, 34,994 balanced DAIGT V2 rows and 53,806 balanced HC3 rows, which is the scale the reported results come from. The smaller 6,000-row sweep from the midterm lives in `notebooks/nlp_final_project_group_02.ipynb` and is not reproduced here.

What this notebook does not do. It does not search the hyperparameter grid. That search already ran, its sixteen transformer configurations per dataset are recorded in `tables/table1_experiments_full.csv`, and the winners are saved as checkpoints. Sections 2 and 3 train the winning configuration, reading the configuration from the deployed checkpoint rather than from a number typed into this file. Section 4 needs no training at all, since it combines probabilities the two checkpoints already wrote.

Runtime. Sections 2 and 3 fine-tune a transformer on tens of thousands of documents. On the RTX 3060 Ti this project used, one run took between 10 and 19 minutes. Set `TRAIN_FROM_SCRATCH = False` in Section 0 to load the saved checkpoints instead and reproduce every reported number in under a minute.

Section 0 --- Environment and paths

Cache locations are pinned before any HuggingFace import, because the root filesystem on the machine this ran on is too small to hold the model cache.

In [ ]:
```
import os

# Must be set before transformers is imported anywhere.
os.environ.setdefault('HF_HOME', '/media/filwel/MLProject1/hf_cache')
os.environ.setdefault('HF_HUB_DISABLE_SYMLINKS', '1')
os.environ.setdefault('PYTORCH_CUDA_ALLOC_CONF', 'expandable_segments:True')
```

```
os.environ.setdefault('TOKENIZERS_PARALLELISM', 'false')

import gc
import json
import re
import shutil
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch

warnings.filterwarnings('ignore')
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True

# Where the raw corpora live. They are large and sit outside the repository.
PROJECT_DIR = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PR
FINAL_DIR   = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PR

PS_DIR      = FINAL_DIR / 'experiments' / 'paper_scale'
WORK_DIR    = PS_DIR / 'work'         # split parquets and index files
RESULTS_DIR = PS_DIR / 'results'      # one JSON per run
PROBS_DIR   = PS_DIR / 'probs'        # per-document probabilities, one NPZ
MODELS_DIR  = PS_DIR / 'models'       # the deployed checkpoints
CKPT_DIR    = Path('/media/filwel/MLProject1/nlp_paper_ckpt')   # scratch, c
for d in (WORK_DIR, RESULTS_DIR, PROBS_DIR, MODELS_DIR, CKPT_DIR):
    d.mkdir(parents=True, exist_ok=True)

# Set False to skip fine-tuning and read the saved checkpoints instead.
TRAIN_FROM_SCRATCH = True

MAX_LEN      = 128     # calibrated in experiments/audit/seqlen_calibration.
EPOCHS       = 5
WARMUP_RATIO = 0.1
PATIENCE     = 2       # early stopping, on validation weighted F1
SPLIT_SEED   = 42      # the partition seed, fixed once and never varied per
TRAIN_SEED   = 42      # initialisation and shuffling seed

MODELS = {'BERT': 'bert-base-uncased', 'DeBERTa': 'microsoft/deberta-v3-base
DATASET_NAMES = {'D1': 'DAIGT V2', 'D2': 'HC3'}

print('torch', torch.__version__, '| cuda', torch.cuda.is_available())
if torch.cuda.is_available():
    print('device', torch.cuda.get_device_name(0),
          '| bf16', torch.cuda.is_bf16_supported())
```

Section 1 --- Data preprocessing

Four steps, in this order.

1.1 Loading. DAIGT V2 arrives as a CSV with a text column and a 0/1 label. HC3 arrives as JSON lines with a list of human answers and a list of ChatGPT answers per question, so it has to be exploded into one row per answer before it is a classification corpus at all.

1.2 Class balancing. Both corpora are imbalanced as published, DAIGT V2 at roughly 61 to 39 and HC3 more mildly. Each is downsampled to the smaller class, so accuracy and weighted F1 cannot be inflated by a majority prior. This is what makes the label-free control in the paper interpretable, since a degenerate all-one-class predictor scores 0.333 weighted F1 on a balanced split rather than something respectable.

1.3 Duplicate-group-aware splitting. HC3 contains 6,118 duplicate rows, 7.16% of the corpus. A plain stratified split puts near-identical answers on both sides of the train/test boundary and the reported score then partly measures memorisation. Rows are grouped by a hash of their normalised content and whole groups are assigned to one partition. Measured effect, from `experiments/audit/verify_paper_claims.py`, is that the group-aware split leaks 0 of 10,732 HC3 test rows while a naive stratified split of the same balanced sample leaks 570, 5.30%. DAIGT V2 leaks nothing either way, and gets the same treatment for consistency.

The split is **72/8/20**, not 80/10/10. Twenty per cent is taken for test, then a tenth of the remaining eighty per cent for validation.

1.4 Two text pipelines, one per model family. The classical models get lowercasing, non-alphabetic removal, stopword removal and lemmatisation. The transformers get raw text, whitespace-normalised only, truncated to 128 tokens by their own tokenisers. Transformers are pretrained on natural text and stripping punctuation and casing removes signal they can use, so applying the classical pipeline to them would handicap them.

In [ ]:
```
import hashlib

from sklearn.model_selection import GroupShuffleSplit


def normalise(t):
    """Whitespace normalisation only. This is all the transformers get."""
    return re.sub(r'\s+', ' ', str(t)).strip()


def content_hash(series):
    """Group key for the duplicate-aware split. Case- and whitespace-insensi
    two answers that differ only in formatting land in the same group."""
    return series.map(lambda t: hashlib.md5(normalise(t).lower().encode()).h


def balance(df, seed=SPLIT_SEED):
```

```
    """Downsample every class to the size of the smallest one."""
    n = int(df['label'].value_counts().min())
    parts = [df[df['label'] == v].sample(n=n, random_state=seed)
             for v in sorted(df['label'].unique())]
    return pd.concat(parts).sample(frac=1, random_state=seed).reset_index(dr


def load_D1():
    """DAIGT V2. 44,868 argumentative essays, human side from the PERSUADE c
    machine side from a mixture of 2023-era generators."""
    raw = pd.read_csv(PROJECT_DIR / 'daigt.csv')
    df = raw[['text', 'label']].dropna()
    df['label'] = df['label'].astype(int)
    del raw
    gc.collect()
    return balance(df)


def load_D2():
    """HC3. Question-answer pairs, one human list and one ChatGPT list per c
    exploded into one row per answer."""
    raw = pd.read_json(PROJECT_DIR / 'hc3.jsonl', lines=True)
    human = raw[['human_answers']].explode('human_answers').rename(
        columns={'human_answers': 'text'})
    human['label'] = 0
    bot = raw[['chatgpt_answers']].explode('chatgpt_answers').rename(
        columns={'chatgpt_answers': 'text'})
    bot['label'] = 1
    df = pd.concat([human, bot], ignore_index=True).dropna()
    df['text'] = df['text'].astype(str)
    del raw, human, bot
    gc.collect()
    return balance(df)


LOADERS = {'D1': load_D1, 'D2': load_D2}


def group_split(df, seed=SPLIT_SEED):
    """72/8/20 train/val/test, never splitting a duplicate-content group."""
    groups = df['hash'].values
    gss1 = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=seed)
    tr_full, te = next(gss1.split(df, df['label'], groups))
    sub = df.iloc[tr_full]
    gss2 = GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=seed)
    tr_rel, val_rel = next(gss2.split(sub, sub['label'], sub['hash'].values)
    idx_tr, idx_val = sub.index.values[tr_rel], sub.index.values[val_rel]
    idx_te = df.index.values[te]
    # The assertion is the point of the whole function, so it stays in.
    g_tr, g_val, g_te = (set(df.loc[idx_tr, 'hash']), set(df.loc[idx_val, 'h
                         set(df.loc[idx_te, 'hash']))
    assert not (g_tr & g_val) and not (g_tr & g_te) and not (g_val & g_te),
        'a duplicate group crossed a split boundary'
    return idx_tr, idx_val, idx_te
```

1.5 Build the splits, or load the ones already built

The split is built **once** and reused by every model and every training seed. Rebuilding it per run would silently change the evaluation set between runs and make the three-seed comparison meaningless.

In [ ]:
```
def build_or_load_splits(tag, rebuild=False):
    data_p = WORK_DIR / f'data_{tag}.parquet'
    split_p = WORK_DIR / f'split_{tag}.npz'
    if not rebuild and data_p.exists() and split_p.exists():
        df = pd.read_parquet(data_p)
        sp = np.load(split_p)
        return df, {'train': sp['train'], 'val': sp['val'], 'test': sp['test

    df = LOADERS[tag]()
    df['hash'] = content_hash(df['text'])
    n_groups = df['hash'].nunique()
    idx_tr, idx_val, idx_te = group_split(df)
    print(f'{tag}  balanced={len(df)}  content_groups={n_groups} '
          f'(duplicate rows={len(df) - n_groups})')
    print(f'   train={len(idx_tr)} val={len(idx_val)} test={len(idx_te)}')
    print(f'   test label balance {df.loc[idx_te, "label"].value_counts().sc
    df[['text', 'label']].to_parquet(data_p, index=True)
    np.savez(split_p, train=idx_tr, val=idx_val, test=idx_te)
    return df[['text', 'label']], {'train': idx_tr, 'val': idx_val, 'test':


DATA, SPLITS = {}, {}
for tag in ('D1', 'D2'):
    DATA[tag], SPLITS[tag] = build_or_load_splits(tag)
    n = {k: len(v) for k, v in SPLITS[tag].items()}
    total = sum(n.values())
    print(f'{tag} {DATASET_NAMES[tag]:9s} total={total:6d}  '
          f'train={n["train"]} ({n["train"]/total:.0%})  '
          f'val={n["val"]} ({n["val"]/total:.0%})  '
          f'test={n["test"]} ({n["test"]/total:.0%})')
```

1.6 The classical text pipeline

Used by the Naive Bayes, logistic regression and SVM baselines the report compares against. Kept here because the report asks for the preprocessing code, and this is the half of it the transformers do not use.

In [ ]:
```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

for pkg in ('punkt', 'punkt_tab', 'stopwords', 'wordnet', 'omw-1.4'):
    try:
        nltk.download(pkg, quiet=True)
```

```
    except Exception as exc:
        print('nltk download skipped for', pkg, exc)

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))


def preprocess_classical(text):
    """Lowercase, drop everything outside [a-z ], tokenise, drop stopwords a
    single characters, lemmatise. Punctuation and casing cannot survive this
    is exactly why the paper's content-only arm uses the same first two step
    text = re.sub(r'[^a-z\s]', ' ', str(text).lower())
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)


demo = DATA['D2']['text'].iloc[0]
print('raw       ', repr(demo[:160]))
print('normalised', repr(normalise(demo)[:160]))
print('classical ', repr(preprocess_classical(demo)[:160]))
```

1.7 Tokenisation for the transformers

Sequence length is fixed at 128 tokens. The consequence is not cosmetic and the paper reports it as a limitation rather than burying it. On a 2,000-row sample of DAIGT V2, 99.7% of documents exceed 128 tokens and the median document keeps only 30.8% of its text. Human essays run about 12.6% longer than machine ones there, so truncation discards proportionally more human content.

In [ ]:
```
from datasets import Dataset
from transformers import AutoTokenizer

_TOKCACHE, _DATACACHE = {}, {}


def get_tokenizer(model_key):
    if model_key not in _TOKCACHE:
        _TOKCACHE[model_key] = AutoTokenizer.from_pretrained(MODELS[model_ke
    return _TOKCACHE[model_key]


def get_tokenized(tag, model_key):
    """Tokenise the three partitions once per (dataset, model) pair and cach
    key = (tag, model_key)
    if key in _DATACACHE:
        return _DATACACHE[key]
    df, splits = DATA[tag], SPLITS[tag]
    tok = get_tokenizer(model_key)
    parts = {}
    for split, idx in splits.items():
        sub = df.loc[idx]
```

```
        ds = Dataset.from_dict({'text': [normalise(t) for t in sub['text']],
                                'labels': [int(v) for v in sub['label']]})
        parts[split] = ds.map(
            lambda b: tok(b['text'], truncation=True, max_length=MAX_LEN),
            batched=True, remove_columns=['text'])
    _DATACACHE[key] = (parts, splits)
    gc.collect()
    return _DATACACHE[key]


tok = get_tokenizer('BERT')
lens = [len(tok(normalise(t))['input_ids']) for t in DATA['D1']['text'].head
lens = np.array(lens)
print(f'DAIGT V2, 2000-row sample, bert-base-uncased tokenizer')
print(f'  median tokens {np.median(lens):.0f}   mean {lens.mean():.1f}')
print(f'  exceeding {MAX_LEN} tokens: {(lens > MAX_LEN).mean():.1%}')
```

Section 2 --- The fine-tuning harness

Sections 3 and 4 both use this. It is written once rather than twice, so BERT and DeBERTa are trained by identical code and any difference between them is a property of the models rather than of two accidentally divergent training loops.

Two details matter for reproducibility. Best-epoch selection is on **validation** weighted F1, never test. And GPU training is not bitwise deterministic here, so re-running the same seed can select a materially different best epoch. The paper reports seed spread as a measured range for the cell it was measured on and rests no claim on it, using paired tests on saved predictions instead.

In [ ]:
```
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_recall_fscore_support)
from transformers import (AutoModelForSequenceClassification,
                          DataCollatorWithPadding, EarlyStoppingCallback,
                          Trainer, TrainingArguments, set_seed)


def weighted_metrics(y, p):
    acc = accuracy_score(y, p)
    pre, rec, f1, _ = precision_recall_fscore_support(
        y, p, average='weighted', zero_division=0)
    return acc, pre, rec, f1


def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc, pre, rec, f1 = weighted_metrics(labels, preds)
    return {'accuracy': acc, 'precision': pre, 'recall': rec, 'f1': f1}


def atomic_write_json(path, obj):
    path = Path(path)
```

```
    tmp = path.with_suffix(path.suffix + '.tmp')
    with open(tmp, 'w') as fh:
        json.dump(obj, fh, indent=2)
        fh.flush()
        os.fsync(fh.fileno())
    os.replace(tmp, path)


def deployed_config(tag, model_key):
    """Read the winning configuration from the checkpoint that is actually d

    This is deliberate. An earlier version of this project reported, per cel
    F1 found anywhere in the hyperparameter grid while the saved weights cam
    different configuration. Two downstream experiments were invalidated by
    the configuration from the checkpoint's own record makes that class of e
    impossible here."""
    p = MODELS_DIR / f'{tag}_{model_key}' / 'run_info.json'
    if not p.exists():
        raise FileNotFoundError(
            f'no deployed checkpoint for {tag}/{model_key} at {p}. '
            'Run experiments/paper_scale/run_full_scale.py first, or set the
            'configuration by hand from tables/table1_experiments_full.csv.'
    r = json.load(open(p))
    return {'lr': r['lr'], 'bs': r['batch_size'], 'wd': r['weight_decay'],
            'key': r['key'], 'recorded_test_f1': r['test']['f1'],
            'recorded_val_f1': r['val']['f1'], 'epochs_run': r['epochs_run']


def train_one(tag, model_key, cfg, seed=TRAIN_SEED, save_model=False):
    """One fine-tuning run at a fixed configuration. Returns the metrics rec
    writes per-document probabilities so downstream paired tests are possibl
    lr, bs, wd = cfg['lr'], cfg['bs'], cfg['wd']
    key = f'full_{tag}_{model_key}_lr{lr:g}_bs{bs}_wd{wd:g}_s{seed}'
    run_dir = CKPT_DIR / key
    parts, splits = get_tokenized(tag, model_key)
    tok = get_tokenizer(model_key)

    set_seed(seed)
    model = AutoModelForSequenceClassification.from_pretrained(
        MODELS[model_key], num_labels=2)
    model.config.id2label = {0: 'human', 1: 'ai'}
    model.config.label2id = {'human': 0, 'ai': 1}

    args = TrainingArguments(
        output_dir=str(run_dir), learning_rate=lr,
        per_device_train_batch_size=bs, per_device_eval_batch_size=64,
        weight_decay=wd, num_train_epochs=EPOCHS,
        warmup_ratio=WARMUP_RATIO, lr_scheduler_type='linear',
        optim='adamw_torch', bf16=torch.cuda.is_bf16_supported(),
        eval_strategy='epoch', save_strategy='epoch', save_total_limit=1,
        load_best_model_at_end=True, metric_for_best_model='eval_f1',
        greater_is_better=True, logging_steps=200,
        seed=seed, data_seed=seed, dataloader_num_workers=0, report_to='none

    trainer = Trainer(
        model=model, args=args, train_dataset=parts['train'],
```

```
        eval_dataset=parts['val'], data_collator=DataCollatorWithPadding(tok
        compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(early_stopping_patience=PATIENCE)])

    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
    t0 = time.time()
    trainer.train()
    train_secs = time.time() - t0

    out = {'key': key, 'dataset': tag, 'dataset_name': DATASET_NAMES[tag],
           'model': model_key, 'checkpoint': MODELS[model_key],
           'lr': lr, 'batch_size': bs, 'weight_decay': wd, 'seed': seed,
           'max_len': MAX_LEN, 'scale': 'full_balanced',
           'n_train': len(splits['train']), 'n_val': len(splits['val']),
           'n_test': len(splits['test']),
           'train_seconds': round(train_secs, 1),
           'epochs_run': int(trainer.state.epoch or 0)}

    probs = {}
    for split in ('val', 'test'):
        pred = trainer.predict(parts[split])
        raw = pred.predictions[0] if isinstance(pred.predictions, tuple) els
        p = torch.softmax(torch.tensor(raw, dtype=torch.float32), dim=-1).nu
        y = np.asarray(pred.label_ids)
        acc, pre, rec, f1 = weighted_metrics(y, p.argmax(1))
        out[split] = {'accuracy': round(acc, 4), 'precision': round(pre, 4),
                      'recall': round(rec, 4), 'f1': round(f1, 4)}
        out[f'{split}_confusion'] = confusion_matrix(y, p.argmax(1)).tolist(
        probs[f'{split}_probs'] = p
        probs[f'{split}_labels'] = y

    np.savez(PROBS_DIR / f'{key}.npz', **probs)
    atomic_write_json(RESULTS_DIR / f'{key}.json', out)

    if save_model:
        mdir = MODELS_DIR / f'{tag}_{model_key}'
        trainer.save_model(str(mdir))
        tok.save_pretrained(str(mdir))
        json.dump(out, open(mdir / 'run_info.json', 'w'), indent=2)

    del trainer, model
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    shutil.rmtree(run_dir, ignore_errors=True)
    print(f'[done] {key}  val_f1={out["val"]["f1"]:.4f}  '
          f'test_f1={out["test"]["f1"]:.4f}  {train_secs/60:.1f} min')
    return out


def load_or_train(tag, model_key):
    """Honour TRAIN_FROM_SCRATCH. Either way the configuration is the deploy
    cfg = deployed_config(tag, model_key)
    if TRAIN_FROM_SCRATCH:
        return train_one(tag, model_key, cfg, save_model=False), cfg
```

```
    rec = json.load(open(RESULTS_DIR / f'{cfg["key"]}.json'))
    print(f'[load] {cfg["key"]}  val_f1={rec["val"]["f1"]:.4f}  '
          f'test_f1={rec["test"]["f1"]:.4f}')
    return rec, cfg
```

Section 3 --- BERT at its best configuration

`bert-base-uncased`, 110M parameters, WordPiece tokeniser.

The grid searched was learning rate in {2e-5, 3e-5}, batch size in {16, 32} and weight decay in {0.01, 0.1}, with the operating point chosen on validation weighted F1. The cell below reads the winner from the deployed checkpoint rather than restating it, then trains that configuration.

One property of this tokeniser matters later. WordPiece splits on punctuation regardless of adjacent whitespace, so `"the answer is simple ."` and `"the answer is simple."` produce identical token identifiers. BERT therefore cannot represent the space-before-punctuation cue that dominates discussion of HC3, and still reaches 0.9916 weighted F1 there. Section 5 shows the check.

In [ ]:
```
BERT_RESULT, BERT_CFG = {}, {}
for tag in ('D1', 'D2'):
    cfg = deployed_config(tag, 'BERT')
    print(f'{tag} {DATASET_NAMES[tag]:9s} best BERT config  '
          f'lr={cfg["lr"]:g}  batch={cfg["bs"]}  weight_decay={cfg["wd"]:g}
          f'(recorded test F1 {cfg["recorded_test_f1"]:.4f})')

for tag in ('D1', 'D2'):
    BERT_RESULT[tag], BERT_CFG[tag] = load_or_train(tag, 'BERT')
```

Section 4 --- DeBERTa, the BERT variant, at its best configuration

`microsoft/deberta-v3-base`, 184M parameters, SentencePiece tokeniser.

DeBERTa-v3 was chosen as the variant for three reasons. Its disentangled attention separates content and position, which is the right inductive bias for a task where the paper's own decomposition shows orthography carries much of the signal. Its ELECTRA-style pretraining is more sample-efficient than masked language modelling at the same parameter budget. And its SentencePiece tokeniser encodes leading whitespace, so unlike WordPiece it can represent the HC3 cue, which makes the two models a useful contrast rather than two draws from the same family.

The same grid, the same selection rule, the same harness.

In [ ]:
```
DEBERTA_RESULT, DEBERTA_CFG = {}, {}
for tag in ('D1', 'D2'):
```

```
    cfg = deployed_config(tag, 'DeBERTa')
    print(f'{tag} {DATASET_NAMES[tag]:9s} best DeBERTa config  '
          f'lr={cfg["lr"]:g}  batch={cfg["bs"]}  weight_decay={cfg["wd"]:g}
          f'(recorded test F1 {cfg["recorded_test_f1"]:.4f})')

for tag in ('D1', 'D2'):
    DEBERTA_RESULT[tag], DEBERTA_CFG[tag] = load_or_train(tag, 'DeBERTa')
```

4.1 The tokeniser contrast, checked rather than asserted

Whether a model can exploit the whitespace cue is testable, so it is tested.

In [ ]:
```
PAIRS = [('the answer is simple .', 'the answer is simple.'),
         ('yes , it does .',        'yes, it does.'),
         ('well ; consider this .', 'well; consider this.')]

for mk in ('BERT', 'DeBERTa'):
    t = get_tokenizer(mk)
    same = sum(t(a)['input_ids'] == t(b)['input_ids'] for a, b in PAIRS)
    print(f'{mk:8s} identical token ids for {same}/{len(PAIRS)} spaced-vs-ur
    print('        ', t.tokenize(PAIRS[0][0]))
    print('        ', t.tokenize(PAIRS[0][1]))
```

Section 5 --- Ensemble

A soft-vote ensemble over the two deployed checkpoints. Predicted class probabilities are mixed as `w * P_BERT + (1 - w) * P_DeBERTa`, and `w` is chosen on the **validation** split only, then applied once to test. Selecting `w` on test would be selecting on the number being reported.

Two honesty notes, both of which the report and paper state rather than hide.

First, the weight sweep is allowed to reach 0 or 1, and it is reported when it does. A weight of 0 means validation selection discarded BERT entirely and the ensemble is not an ensemble.

Second, an ensemble that scores higher than its best member has not necessarily beaten it. The comparison is made on the same test documents, so it gets the same paired test every other comparison in this project uses, exact-binomial McNemar on the discordant predictions plus a paired bootstrap interval on the error difference.

In [ ]:
```
from scipy.stats import binomtest


def load_probs(key):
    z = np.load(PROBS_DIR / f'{key}.npz')
    return {k: z[k] for k in z.files}


def paired_test(y, pred_a, pred_b, n_boot=10000, seed=SPLIT_SEED):
```

```
    """McNemar exact plus a paired bootstrap on the error difference, a minu
    rng = np.random.default_rng(seed)
    wa, wb = pred_a != y, pred_b != y
    b = int((~wa & wb).sum())
    c = int((wa & ~wb).sum())
    p = binomtest(b, b + c, 0.5).pvalue if (b + c) else 1.0
    idx = rng.integers(0, len(y), size=(n_boot, len(y)))
    boot = wa[idx].mean(1) - wb[idx].mean(1)
    lo, hi = np.percentile(boot, [2.5, 97.5])
    return {'b': b, 'c': c, 'p': float(p),
            'diff_pp': float((wa.mean() - wb.mean()) * 100),
            'ci_lo_pp': float(lo * 100), 'ci_hi_pp': float(hi * 100)}


WEIGHTS = np.round(np.arange(0.0, 1.0001, 0.05), 2)
ENSEMBLE = {}

for tag in ('D1', 'D2'):
    pb = load_probs(BERT_CFG[tag]['key'])
    pdb = load_probs(DEBERTA_CFG[tag]['key'])
    assert np.array_equal(pb['val_labels'], pdb['val_labels'])
    assert np.array_equal(pb['test_labels'], pdb['test_labels'])
    yv, yt = pb['val_labels'], pb['test_labels']

    val_f1 = []
    for w in WEIGHTS:
        mix = w * pb['val_probs'] + (1 - w) * pdb['val_probs']
        val_f1.append(weighted_metrics(yv, mix.argmax(1))[3])
    best_w = float(WEIGHTS[int(np.argmax(val_f1))])

    ens_pred = (best_w * pb['test_probs'] + (1 - best_w) * pdb['test_probs']
    acc, pre, rec, f1 = weighted_metrics(yt, ens_pred)

    members = {'BERT': pb['test_probs'].argmax(1),
               'DeBERTa': pdb['test_probs'].argmax(1)}
    mem_f1 = {k: weighted_metrics(yt, v)[3] for k, v in members.items()}
    stronger = max(mem_f1, key=mem_f1.get)
    pt = paired_test(yt, ens_pred, members[stronger])

    ENSEMBLE[tag] = {'weight_bert': best_w, 'weight_deberta': round(1 - best
                     'test_f1': round(f1, 4), 'test_accuracy': round(acc, 4)
                     'member_f1': {k: round(v, 4) for k, v in mem_f1.items()
                     'stronger_member': stronger, 'paired': pt,
                     'degenerate': best_w in (0.0, 1.0),
                     'confusion': confusion_matrix(yt, ens_pred).tolist()}

    print(f'{tag} {DATASET_NAMES[tag]}')
    print(f'   weight on BERT {best_w:.2f}, on DeBERTa {1-best_w:.2f}'
          + ('   DEGENERATE, collapsed onto one member' if best_w in (0.0, 1
    print(f'   members  BERT {mem_f1["BERT"]:.4f}   DeBERTa {mem_f1["DeBERTa
    print(f'   ensemble {f1:.4f}   against stronger member {stronger} '
          f'{mem_f1[stronger]:.4f}')
    print(f'   paired   McNemar p={pt["p"]:.4g}  error diff {pt["diff_pp"]:+
          f'95% CI [{pt["ci_lo_pp"]:+.3f}, {pt["ci_hi_pp"]:+.3f}]')
    verdict = ('improvement is not distinguishable from zero'
               if pt['ci_lo_pp'] <= 0 <= pt['ci_hi_pp']
```

```
                else 'difference excludes zero')
    print(f'   verdict  {verdict}\n')
```

5.1 The weight sweep, plotted

Where the curve is flat, the ensemble has nothing to gain from mixing.

In [ ]:
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
for ax, tag in zip(axes, ('D1', 'D2')):
    pb = load_probs(BERT_CFG[tag]['key'])
    pdb = load_probs(DEBERTA_CFG[tag]['key'])
    yv = pb['val_labels']
    curve = [weighted_metrics(yv, (w * pb['val_probs']
                                   + (1 - w) * pdb['val_probs']).argmax(1))[
             for w in WEIGHTS]
    ax.plot(WEIGHTS, curve, marker='o', ms=3, color='#4477AA')
    ax.axvline(ENSEMBLE[tag]['weight_bert'], color='#EE6677', ls='--',
               label=f'chosen w = {ENSEMBLE[tag]["weight_bert"]:.2f}')
    ax.set_title(f'{tag}  {DATASET_NAMES[tag]}')
    ax.set_xlabel('weight on BERT (1 minus w on DeBERTa)')
    ax.set_ylabel('validation weighted F1')
    ax.legend()
plt.tight_layout()
plt.show()
```

Section 6 --- Summary table

Every figure below comes from the runs above. Nothing is typed in.

In [ ]:
```
rows = []
for tag in ('D1', 'D2'):
    for mk, res in (('BERT', BERT_RESULT[tag]), ('DeBERTa', DEBERTA_RESULT[t
        rows.append({'dataset': DATASET_NAMES[tag], 'model': mk,
                     'lr': res['lr'], 'batch': res['batch_size'],
                     'weight_decay': res['weight_decay'],
                     'val_f1': res['val']['f1'], 'test_f1': res['test']['f1'
                     'test_acc': res['test']['accuracy'],
                     'test_err_pct': round((1 - res['test']['accuracy']) * 1
    e = ENSEMBLE[tag]
    rows.append({'dataset': DATASET_NAMES[tag],
                 'model': f'Ensemble (w_BERT={e["weight_bert"]:.2f})',
                 'lr': None, 'batch': None, 'weight_decay': None, 'val_f1':
                 'test_f1': e['test_f1'], 'test_acc': e['test_accuracy'],
                 'test_err_pct': round((1 - e['test_accuracy']) * 100, 2)})

summary = pd.DataFrame(rows)
print(summary.to_string(index=False))
summary.to_csv(FINAL_DIR / 'tables' / 'notebook_submission_summary.csv', inc
```

What these numbers do and do not say

DeBERTa is the stronger model on both corpora, and the margin is small on DAIGT V2 and large on HC3. The ensemble does not reliably improve on it. On DAIGT V2 the mixed model scores slightly higher than either member but the paired interval on the error difference contains zero, so the improvement is not distinguishable from noise on this test set. On HC3 validation selection put zero weight on BERT, so the ensemble is DeBERTa and reports DeBERTa's score.

The report treats the ensemble as a discussion point rather than a headline result, for that reason. Reporting a soft-vote ensemble as an improvement when its interval contains zero, or when its selected weight has discarded one member, would be reading a rounding difference as a finding.

Two limits carry into every number above. Both transformers see at most 128 tokens, which is roughly a third of a mean DAIGT V2 essay, so these are results about essay openings on that corpus. And these are single-seed figures at the deployed configuration. Three-seed ranges for the same cells are recorded in `experiments/audit/paper_claim_verification.json`, and no comparison in the paper rests on a seed range.